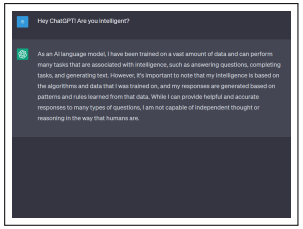
Lecture 02
Formal Foundations of Computing

Algorithms, AI & Data Science II

Prof. Dr. Ingo Scholtes
Chair of Machine Learning for Complex Networks
Center for Artificial Intelligence and Data Science (CAIDAS)
Julius-Maximilians-Universität Würzburg
ingo.scholtes@uni-wuerzburg.de

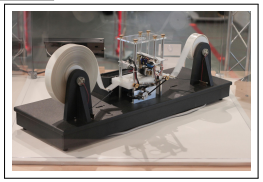
Motivation

- deep understanding of algorithms and artificial intelligence requires to **formalize "computation"**
- how can we **formally state a problem** such that a machine can given an answer?
- we explore **formal languages** and study **automata models** to address **word decision problems**
- we define **Turing machines** and investigate **computability and "machine intelligence"**
- we start with basic **mathematical foundations of computer science**
  1. set theory and set operations
  2. relations and functions
  3. notational conventions



response from large language model ChatGPT

image credit: OpenAI screenshot, taken on 12/04/2023



Physical model of a **Turing machine**

image credit: Rocky Acosta, Wikimedia Commons, CC-BY-SA

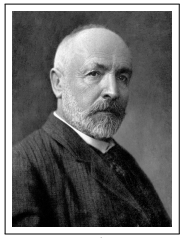
Notes:

- **L02: Formal Foundations of Computing** 30.04.2025
- **Educational objective:** We introduce foundational mathematical concepts like sets and relations, study formal languages and grammars and introduce the Chomsky hierarchy of formal languages.
  - Basic Set Theory
  - Relations and Functions
  - Formal Languages and Grammars
  - Chomsky Hierarchy of Formal Languages
- **Handout version:** *April 30, 2025*

Notes:

Set theory

- **set theory** = mathematical study of relationships between **collections of objects**
- important foundation of discrete mathematics, formal logics and computer science → G Cantor, 1874
- sets are **unordered** collections of **distinguishable** objects
- sets can be **finite or infinite**
- we can unambiguously determine whether an **object is element of a set or not**



Georg Cantor (1845 – 1918)

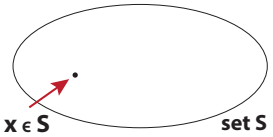


image credit: Wikimedia Commons, public domain

Foundational set theory

definitions

- A **set** is an unordered collection of distinguishable objects, which are called **elements** or **members** of the set.
- For any object x we can unambiguously determine whether it is element of a set S. If x is member of S we write $x \in S$ and $x \notin S$ otherwise.
- The **cardinality** $|S|$ is defined as the number of elements in S.
- We denote the **empty set** (without elements) as $\emptyset$ with $|\emptyset| = 0$.
- We can define sets in (finite or infinite) **enumeration notation**, i.e. $S = \{e_1, e_2, e_3, \ldots\}$.
- We can define sets in **set-builder notation**, i.e. $\{x|p(x)\}$ where p is a condition (or logical predicate) that holds for all members $x \in S$
- S_1 **is subset of** S_2 ($S_1 \subseteq S_2$) iff for each $x \in S_1$ we have $x \in S_2$
- S_1 **and** S_2 **are equal** ($S_1 = S_2$) iff for each $x \in S_1$ we have $x \in S_2$ and vice-versa, i.e. $S_1 \subseteq S_2$ and $S_2 \subseteq S_1$
- S_1 **is proper subset of** S_2 ($S_1 \subset S_2$) iff for each $x \in S_1$ we have $x \in S_2$ and $S_1 \neq S_2$

examples

- $S = \{1, \text{'Cat'}, 42.5\}$ is a set with $|S| = 3$
- $S = \{1, 2, 3, 2\}$ is not a valid set
- $\mathbb{N} = \{1, 2, 3, 4, \ldots\}$ is an (infinite) set
- $\mathbb{N} = \{x | x \text{ is natural number }\}$
- $S_1 = \{1, 2, 3\} \subseteq \{1, 2, 3, 4\} = S_2$
- $\emptyset \subseteq S$ for any set S
- $S_1 = \{1, 2, 3\} = \{3, 2, 1\} = S_2$
- $S_1 = \{1, 2, 3\} \subset \{1, 2, 3, 4\} = S_2$

Notes:

Notes:

Set operations, complement, and power set

definitions

Let S_1 and S_2 be arbitrary sets. We define the following binary operations:

- $S_1 \cup S_2 := \{x | x \in S_1 \text{ or } x \in S_2\}$ **union**
- $S_1 \cap S_2 := \{x | x \in S_1 \text{ and } x \in S_2\}$ **intersection**
- $S_1 - S_2 := \{x | x \in S_1 \text{ and } x \notin S_2\}$ **difference**
- $S_1 \triangle S_2 := (S_1 \cup S_2) - (S_2 \cap S_2) = \{x | x \in S_1 \text{ xor } x \in S_2\}$ **symmetric difference**
- $S_1 \times S_2 := \{(x, y) | x \in S_1 \text{ and } y \in S_2\}$ **product**

- for given set S we define **complement** S^C (relative to some "universe") as the set of all elements in the universe that are not element of S
- for given set S we define **power set** 2^S as the set of all subsets of S, i.e. $2^S := \{S' : S' \subseteq S\}$

examples

Let $S_1 = \{1, 2, 3\}$ and $S_2 = \{2, 3, 4, 5\}$.

- $S_1 \cup S_2 = \{1, 2, 3, 4, 5\}$
- $S_1 \cap S_2 = \{2, 3\}$
- $S_1 - S_2 = \{1\}$
- $S_1 \triangle S_2 = \{1, 4, 5\}$
- $S_1 \times S_2 = \{(1,2), (1,3), (1,4), (1,5), (2,2), (2,3), (2,4), (2,5), (3,2), (3,3), (3,4), (3,5)\}$

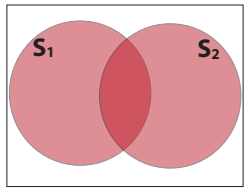
  with $|S_1 \times S_2| = 12$
- For $S = \{1, 2, 3\}$ we have

  $2^S = \{\emptyset, \{1\}, \{2\}, \{3\}, \{1,2\}, \{1,3\}, \{2,3\}, \{1,2,3\}\}$

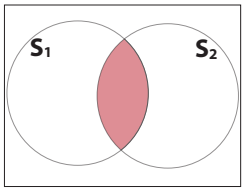
  with $|2^S| = 2^{|S|} = 8$

Venn diagrams

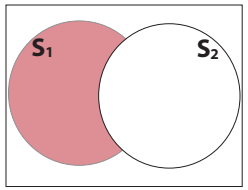
we can use **Venn diagrams** to visually represent operations on sets



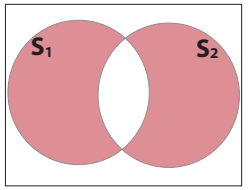
$S_1 \cup S_2$



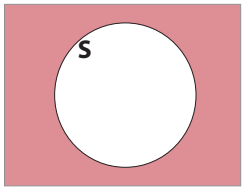
$S_1 \cap S_2$



$S_1 - S_2$



$S_1 \triangle S_2$



S^C

Notes:

Notes:

Relations

- elements of sets often exhibit **relationships** that we can capture mathematically

definition

- Let S_1 and S_2 be arbitrary sets. A (binary) relation R between S_1 and S_2 is a subset of the product set $S_1 \times S_2$, i.e. $R \subseteq S_1 \times S_2$ and $R \in 2^{S_1 \times S_2}$
- Instead of $(a, b) \in R$ we can write $R(a, b)$ or aRb.

- relations $R \subseteq S_1 \times S_2$ can be between elements of **different sets** $S_1 \neq S_2$
- we can define relations $R \subseteq S \times S$ between elements of a **single set**, in which case we call R a **relation on** S

examples

- For $S_1 = \{\text{Peter}, \text{Anna}\}$ and $S_2 = \{\text{C++}, \text{python}\}$

$$R = \{(\text{Peter}, C++), (\text{Anna}, \text{python}), (\text{Anna}, \text{C++})\}$$

  is a relation
- $\emptyset$ is an (empty) relation between any pair of sets
- comparator $<$ is a relation on natural numbers $\mathbb{N}$ with $(2, 5) \in <$ and $(5, 2) \notin <$
- $R = \{(x, y) \in \mathbb{N} \times \mathbb{N} | y = x^2\}$ is a relation on natural numbers $\mathbb{N}$

Properties of relations

definitions

- relation R on S is **reflexive** iff xRx for any $x \in S$
- relation R on S is **symmetric** iff for all $x, y \in S$ with xRy we also have yRx
- relation R on S is **transitive** iff for all $x, y, z \in S$ with xRy and yRz we also have xRz
- a relation R between sets X, Y is called **functional** (or right-unique) iff for all $x \in X$ and $y, y' \in Y$ with $R(x, y)$ and $R(x, y')$ we have $y = y'$
- relation R between X, Y is called **injective** (or left-unique) iff for all $x, x' \in X$ and $y \in Y$ with $R(x, y)$ and $R(x', y)$ we have $x = x'$

- we call functional relation $R \subseteq X \times Y$ a **function** $R : X \to Y$ and write $R(x) = y$ instead of $R(x, y)$
- we call a functional relation $R \subseteq X \times Y$ that is injective **one-to-one mapping**

examples

- comparator $<$ on $\mathbb{N}$ is neither reflexive nor symmetric
- comparator $\leq$ on $\mathbb{N}$ is reflexive but not symmetric
- $<$ and $\leq$ on $\mathbb{N}$ are transitive
- $R = \{(x, x), (y, y), (z, z), (y, z), (z, y)\}$ is reflexive, symmetric, and transitive
- $<$ and $\leq$ on $\mathbb{N}$ are not functional
- $R(x) = x^2$ is functional
- $R(x) = x^2$ is not injective
- $R(x) = x + 1$ is a one-to-one mapping that maps every $x \in \mathbb{N}$ to $x + 1$

Notes:

Notes:

Operations on relations

- Let $R \subseteq S_1 \times S_2$ be a relation. The **inverse of** R is defined as

$$R^{-1} := \{(y, x) \in S_2 \times S_1 | (x, y) \in R\}$$

- Let $R_1 \subseteq S_1 \times S_2$ and $R_2 \subseteq S_3 \times S_4$ be relations. The **inner product of** R_1 **and** R_2 is defined as

$$R_1 \otimes R_2 := \{((a, c), (b, d)) \in (S_1 \times S_3) \times (S_2 \times S_4) | aR_1b \text{ and } cR_2d\}$$

- Let $R_1 \subseteq S_1 \times S_2$ and $R_2 \subseteq S_2 \times S_3$ relations. The **composition of** R_1 **and** R_2 is defined as

$$R_1 \circ R_2 := \{(a, c) \in (S_1 \times S_3) | aR_1b \text{ and } bR_2c \text{ for some } b\}$$

Group Exercise 02-01

1. Consider the "smaller than" relation $<$ on $\mathbb{R}$ and let $<^{-1}$ be the inverse of $<$. Provide a set builder definition for the elements in the inner product $< \otimes <^{-1}$. Give an example for elements of the resulting relation.

2. Prove that $(R \cup S)^{-1} = R^{-1} \cup S^{-1}$

Notes:

Notes:

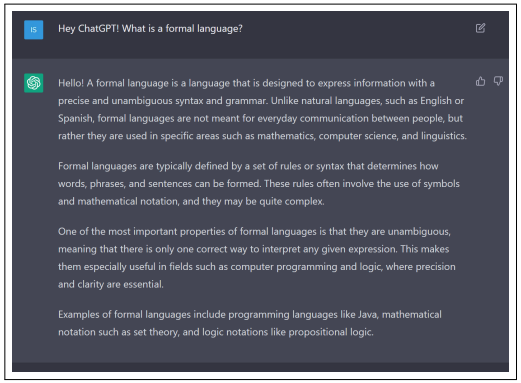
Languages and computers

- humans use **natural languages** (like German, English, French) to communicate
- in natural language, meaning is often **ambiguous or depends on context**, e.g.

  "She saw the man with the telescope."
- to communicate with machines, we (generally) **formalize languages** such that the meaning of statements is precisely defined

examples of formal languages

1. programming languages (C, C++, python)
2. arithmetic expressions
3. Boolean algebra → L03 - Propositional Logic and Boolean Algebra



output of large language model **ChatGPT**

image credit: OpenAI screenshot, created 29/03/2023

What is a formal language?

definition

Let Σ be a **finite alphabet**, i.e. a set of symbols.

- We call a finite sequence $w \in \Sigma^i$ of i symbols a **word** with length $|w| := i$. We denote the set of all words of length i as Σ^i.
- We call ϵ the **empty word** (with length zero).
- We use w^R to denote the word that we get by reversing the sequence of symbols.
- With $\Sigma^0 = \{\epsilon\}$ we recursively define

$$\Sigma^{i+1} := \{w | w = xy \text{ with } x \in \Sigma^i \text{ and } y \in \Sigma\}$$

- We define $\Sigma^* := \bigcup_{i \geq 0} \Sigma^i$ and $\Sigma^+ := \bigcup_{i > 0} \Sigma^i$
- A **formal language** $L \subseteq \Sigma^*$ is a (possibly infinite) set of words over alphabet Σ.

examples

- $L = \{aa, bb, abba\}$ is a (finite) formal language over alphabet $\Sigma = \{a, b\}$
- $L = \{x \in \Sigma^* | x = x^R\}$ is an (infinite) formal language of palindroms over alphabet $\Sigma = \{a, b\}$
- $L = \{x | x \text{ is syntactically correct python program}\}$ is a formal language over the alphabet of ASCII characters

how can we define **syntax** ("$\sigma\upsilon\nu - \tau\alpha\xi$") of words of a formal language L?

Notes:

Notes:

Grammars

- we can use **grammars** to define formal languages based on **production rules**

definition

A **grammar** G is a four-tuple $G = (V, \Sigma, P, S)$ where

- V is a finite set of **variables** used in production rules,
- Σ is a finite alphabet of **terminal symbols** with $V \cap \Sigma = \emptyset$,
- P is a finite set of **production rules** with $P \subseteq (V \cup \Sigma)^+ \times (V \cup \Sigma)^*$, and
- $S \in V$ is a **start symbol**.

For $(X, Y) \in P$ we write $X \to Y$ to denote that X can be substituted by Y.
We use $X \to^* Y$ to denote that there is a (possibly transitive) application of production rules that allows to substitute X by Y.
We say that $L(G) = \{w \in \Sigma^* | S \to^* w\}$ is the **language generated by grammar** G.

- P is a relation that is **not necessarily functional**, i.e. we can have $X \to Y \in P$ and $X \to Y' \in P$, which we denote as $X \to Y \mid Y'$

example

$G = (\{A, B\}, \{a, b\}, P, S)$ with

$$P = \{S \to AB,\\ A \to aA \mid \epsilon,\\ B \to bB \mid \epsilon \}$$

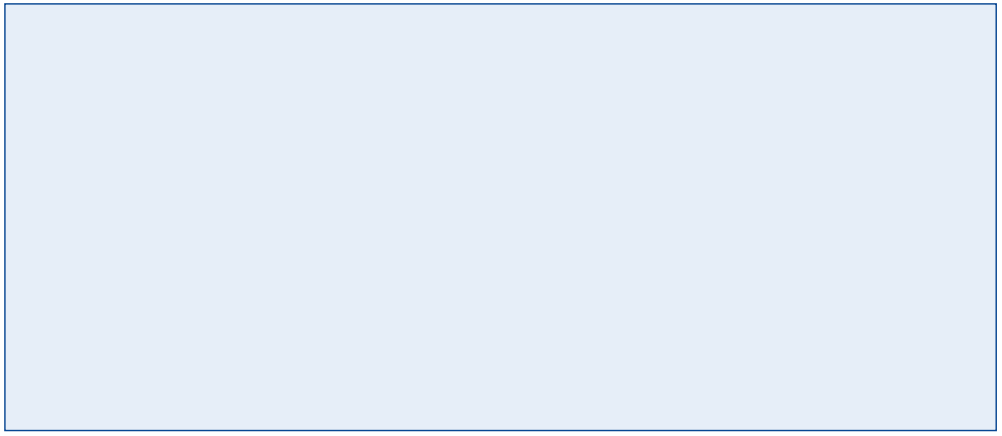
Starting with S we can apply the following substitutions:

$$S \to AB\\ AB \to aAB\\ aAb \to aaAB\\ aaAB \to aaAbB\\ aaAbB \to aabB\\ aabB \to aab$$

Due to $S \to^* aab$ we have $aab \in L(G)$.

Group Exercise 02-02

Write a grammar for the formal language $L = \{a^n b^n c^n | n \geq 1\}$. Explain how the production rules are used to produce the word *aaabbbccc*.



Notes:

Notes:

Chomsky hierarchy

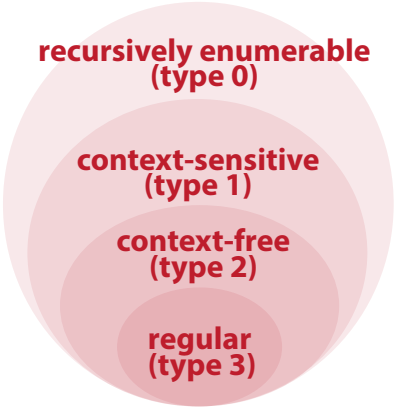
- we **categorize grammars into a hierarchy** based on constraints that must be satisfied by production rules

definition

Let $G = (V, \Sigma, P, S)$ be a grammar.

- Every grammar G is **recursively enumerable (type 0)**
- Grammar G is **context-sensitive (type 1)**, iff for all production rules $w_1 \to w_2 \in P : |w_1| \leq |w_2|$ (or rule has the form $S \to \epsilon$)
- Grammar G of type 1 is **context-free (type 2)**, iff for all rules $w_1 \to w_2 \in P : w_1 \in V$
- Grammar G of type 2 is **regular (type 3)**, iff for all rules $w_1 \to w_2 \in P : w_2 \in \Sigma \cup \Sigma V \cup \{\epsilon\}$

We say that a formal language $L \subseteq \Sigma^*$ is of type 0, 1, 2 or 3 iff a type 0, 1, 2 or 3 grammar G exists with $L(G)$, respectively.



Venn diagram of Chomsky hierarchy

- higher type = stricter rules = **less expressive**

Group Exercise 02-03

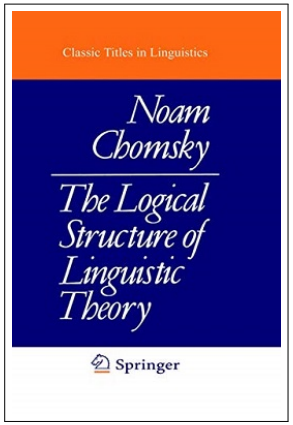
Consider the grammar from group exercise 02-02 and determine its type according to the Chomsky hierarchy.

Notes:

Notes:

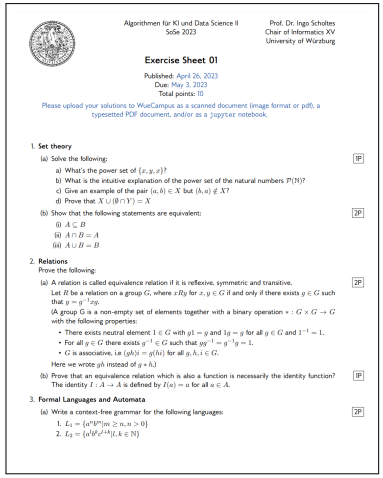
In summary …

- we have covered **formal foundations of computing**
- we introduced **formal languages**, which are defined as (possibly infinite) sets of words over finite alphabet
- we can specify the syntax of formal languages based on **grammars with production rules**
- we use the **Chomsky hierarchy** to categorize grammars (and the languages that they produce) based on their production rules
- formal languages allow us to formalize **which problems are decidable** by an algorithm
- how can we **algorithmically decide** whether a given word w is in a language L?



Exercise sheet 01

- **first exercise sheet** will be released next Monday
- solutions are due **May 12th** (via WueCampus)
- present your solution to earn bonus points



Algorithmen für KI und Data Science II
SoSe 2023

Prof. Dr. Ingo Scholtes
Chair of Informatics XV
University of Würzburg

Exercise Sheet 01

Published: April 26, 2023
Due: May 3, 2023
Total points: 10

Please upload your solutions to WueCampus as a scanned document (image format or pdf), a typesetted PDF document, and/or as a Jupyter notebook.

1. **Set theory**
   (a) Solve the following: [1P]
   a) What's the power set of $\{x, y, x\}$?
   b) What is the intuitive explanation of the power set of the natural numbers $\mathcal{P}(\mathbb{N})$?
   c) Give an example of the pair $(a, b) \in X$ but $(b, a) \notin X$?
   d) Prove that $X \cup (\emptyset \cap Y) = X$
   (b) Show that the following statements are equivalent: [2P]
   (i) $A \subseteq B$
   (ii) $A \cap B = A$
   (iii) $A \cup B = B$
2. **Relations**
   Prove the following:
   (a) A relation is called equivalence relation if it is reflexive, symmetric and transitive. [2P]
   Let R be a relation on a group G, where $x R y$ for $x, y \in G$ if and only if there exists $g \in G$ such that $y = g^{-1} x g$.
   (A group G is a non-empty set of elements together with a binary operation $\kappa : G \times G \to G$ with the following properties:
   - There exists neutral element $1 \in G$ with $g1 = g$ and $1g = g$ for all $g \in G$ and $1^{-1} = 1$.
   - For all $g \in G$ there exists $g^{-1} \in G$ such that $gg^{-1} = g^{-1}g = 1$.
   - G is associative, i.e $(gh)i = g(hi)$ for all $g, h, i \in G$.

   Here we wrote gh instead of $g * h$.)
   (b) Prove that an equivalence relation which is also a function is necessarily the identity function? [1P]
   The identity $I : A \to A$ is defined by $I(a) = a$ for all $a \in A$.
3. **Formal Languages and Automata**
   (a) Write a context-free grammar for the following languages: [2P]
   1. $L_1 = \{a^n b^m | m \geq n, n > 0\}$
   2. $L_2 = \{a^l b^k c^{l+k} | l, k \in \mathbb{N}\}$

Notes:

Notes:

Self-study questions

1. Explain the difference between a relation and a function.
2. Give an example for a relation R on $\mathbb{N}$ that is reflexive, symmetric, and transitive.
3. Show that your relation R from above defines a partition of $\mathbb{N}$, i.e. a partition of $\mathbb{N}$ in pair-wise non-overlapping subsets of $\mathbb{N}$.
4. Give an example for a relation R that is not functional but injective.
5. Provide the formal definition of a grammar. What is the difference between V and Σ in the definition?
6. Give an example for a grammar where the production rules never terminate (thus not a language without any words).
7. Give a grammar for the language of all bit strings that correspond to even natural numbers.
8. Using the production rules of the previous question, test whether a large language model like ChatGPT can correctly derive the resulting language.
9. What is the difference between a content-sensitive and context-free language?
10. Give an example for a language that is regular.
11. Write a grammar for the language of correctly bracketed arithmetic formulas with arithmetic operations $+$ and $*$. For simplicity consider formulas with two variables a, b only.

References

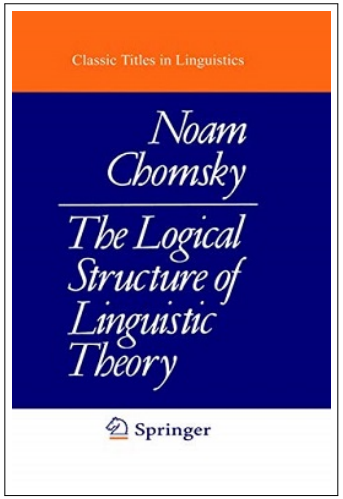
reading list

- G Cantor: **Ueber eine Eigenschaft des Inbegriffes aller reellen algebraischen Zahlen**, Journal über die Reine und Angewandte Mathematik, 1874
- N Chomsky: **Three models for the description of language**, 1956
- N Chomsky: **The Logical Structure of Linguistic Theory**, 1975

acknowledgments

some examples are based on the following books and lectures:

- U Schöning: **Theoretische Informatik - kurzgefasst**, Spektrum Akadem. Verlag, 2001
- J Esparza: **Automata theory: An algorithmic approach**, lecture notes, 2020



Notes:

Notes: